Análise de Eficácia de Testes com Teste de Mutação

Disciplina: **Testes de Software**

Islayder Jackson Ribeiro de Oliveira
Matrícula: 859552

13 de outubro de 2026

Projeto e rastreabilidade

Biblioteca JavaScript com 50 operações matemáticas.
Fork: github.com/Islayder/operacoes-mutante
Revisão dos testes: e32966943f9b

Resultado verificado: 73,71% para 97,65%.
149 testes aprovados; código original preservado.

Limitação declarada: a meta estrita superior a 98% permanece pendente. Cinco mutantes equivalentes no domínio matemático previsto foram mantidos no resultado oficial.

1. Análise inicial e método

A biblioteca de 50 funções foi recuperada do commit c10621a4fe20. Os 50 testes originais passaram antes das melhorias. A instrumentação contabiliza 53 funções porque inclui callbacks. A implementação e a suíte original foram preservadas.

Ambiente: Node 24.19.0, npm 11.17.0, Jest 29.7.0 e StrykerJS/Jest runner 10.0.0, Windows x64. Foram usados os operadores padrão em todo src/operacoes.js, cobertura por teste e duas instâncias. Dependências fixadas no lockfile.

Métrica	Inicial	Final
Testes aprovados	50 / 50	149 / 149
Instruções	85,41%	100,00%
Ramos	58,82%	100,00%
Funções	100,00%	100,00%
Linhas	98,64%	100,00%
Mutation Score oficial	73,71%	97,65%

A cobertura de linhas de 98,64% coexistia com 56 mutantes não detectados. Executar uma linha não garante exercitar seus dois ramos nem distinguir um resultado correto de outro incorreto. A cobertura inicial de ramos, 58,82%, já indicava lacunas.

2. Mutante 11 — mensagem de erro em divisao

Arquivo: src/operacoes.js:8. O StringLiteral substituiu a mensagem por uma string vazia. O teste original usava apenas **toThrow()**: qualquer exceção satisfazia a asserção. O novo teste exige **toThrow(new Error("Divisão por zero não é permitida."))**, verificando a mensagem existente.

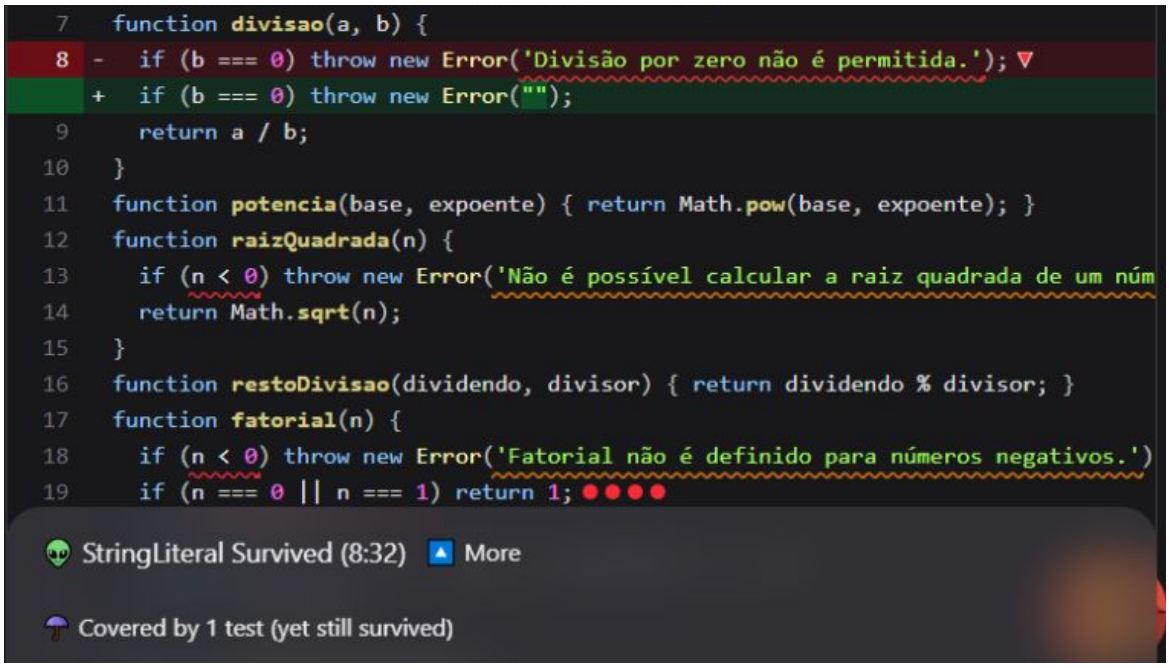


Figura 1. Captura real do HTML inicial: mensagem apagada e estado Survived. Na execução final, o ID 11 foi Killed.

3. Mutante 183 — igualdade em isMaiorQue

src/operacoes.js:104. O EqualityOperator trocou **a > b** por **a >= b**. O único caso original, (10, 5), retorna true em ambas as versões. A fronteira de igualdade estava ausente. A asserção **expect(isMaiorQue(5, 5)).toBe(false)** distingue os comportamentos e matou o mutante.

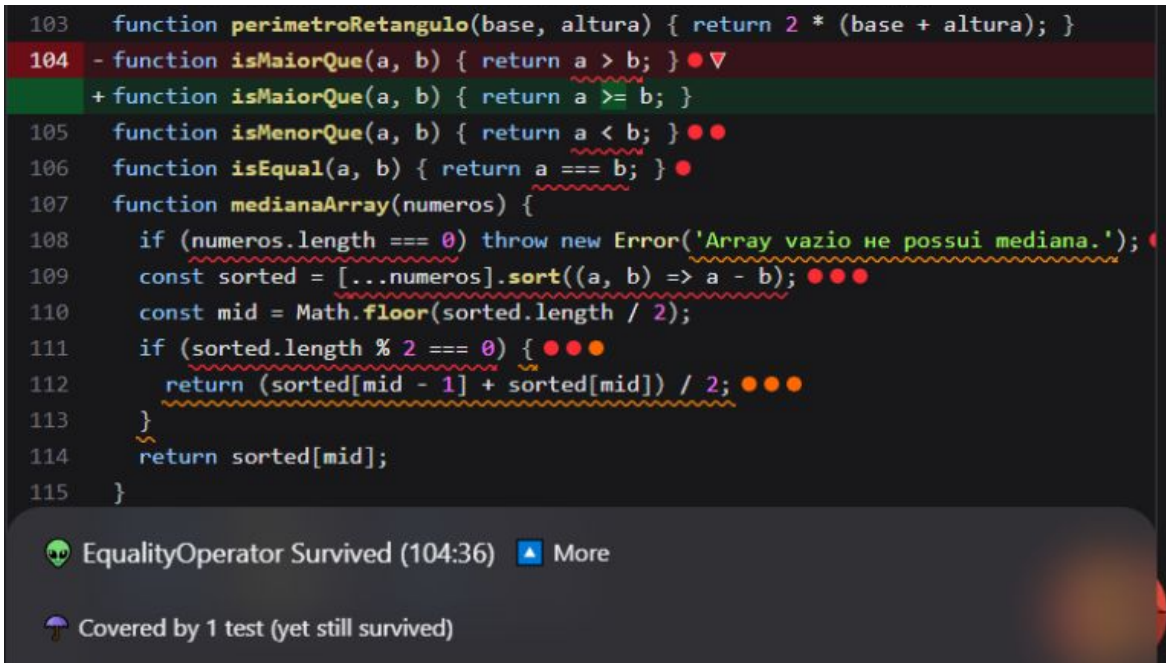


Figura 2. Alteração de > para >= no relatório inicial; ID 183: Survived → Killed.

4. Mutante 200 — ordenação em medianaArray

src/operacoes.js:109. O MethodExpression removeu **.sort((a, b) => a - b)**, mantendo apenas a cópia do array. A entrada original [1, 2, 3, 4, 5] já estava ordenada. Os novos casos **[20, 3, 1] → 3** e **[30, 1, 20, 2] → 11** exigem ordenação numérica; o segundo também cobre a mediana de tamanho par.

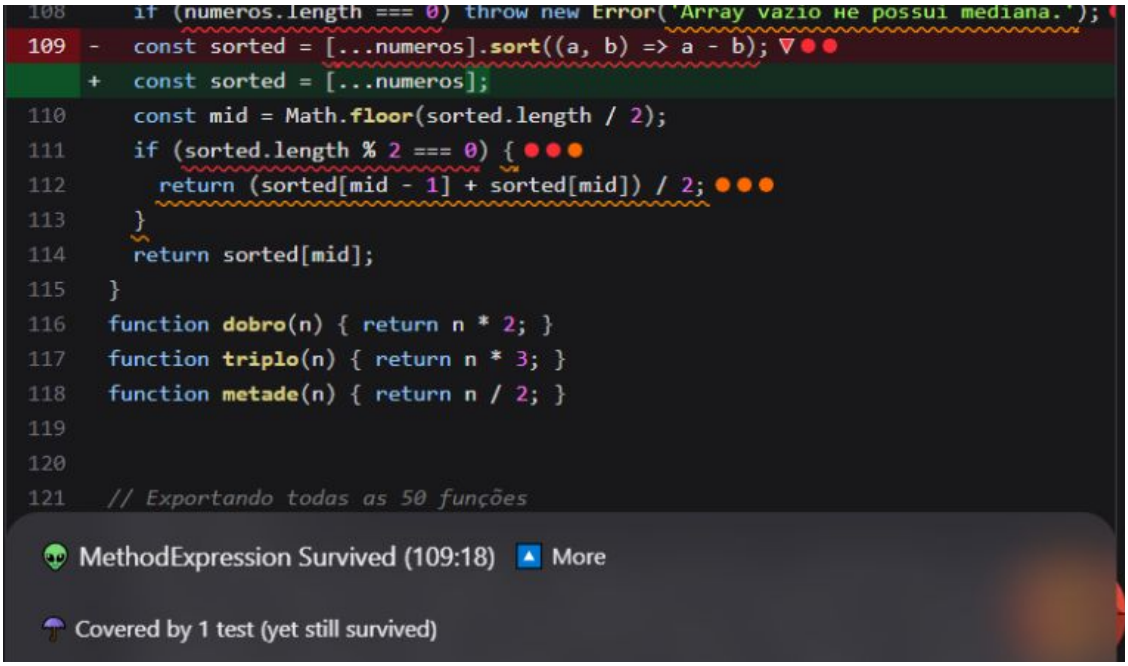


Figura 3. Ordenação removida no HTML original; ID 200: Survived → Killed. Os JPGs completos também acompanham a entrega.

5. Soluções implementadas e validação

Foram adicionados 99 testes em test/operacoes.adicionais.test.js: zero, negativos, erros com mensagens, arrays vazios, primos e compostos, conversões não nulas, igualdade, mediana par/ímpar e entradas desordenadas. Resultados exatos usam toBe; aproximações usam toBeCloseTo com dez casas. A mediana também é verificada quanto à preservação da entrada.

Os limites de clamp incluem os zeros IEEE 754: **clamp(-0, 0, 10)** preserva -0 e **clamp(0, -10, -0)** preserva +0. toBe distingue esses valores e detecta as trocas de < e > por comparações inclusivas.

Estado dos 213 mutantes	Inicial	Final
Mortos por falha de teste	154	202
Timeouts (também detectados)	3	6
Sobreviventes	44	5
Sem cobertura	12	0
Ignorados / erros	0 / 0	0 / 0

Score = (Killed + Timeout) / total válido. Inicial: 157/213 = 73,71%; final: 208/213 = 97,65%, melhora de **23,94 pontos percentuais**. Duas execuções de cada etapa reproduziram as mesmas contagens. As três mutações analisadas foram mortas; a cobertura final atingiu 100% nas quatro métricas.

6. Cinco equivalentes e a meta não atingida

IDs 31, 32, 33 e 35 — fatorial, linha 19. Desabilitar todo ou parte de **n === 0 || n === 1**, ou trocar || por &&, preserva o resultado. Para 0 e 1, o acumulador começa em 1 e o laço iniciado em 2 não executa; para outros números, o retorno antecipado já não ocorre.

ID 140 — produtoArray, linha 84. Retirar o retorno antecipado de array vazio leva a **[].reduce(..., 1)**, que também retorna 1. A equivalência vale para arrays numéricos comuns com reduce nativo; não se generaliza a métodos sobrescritos ou Proxies.

Mantidos esses cinco no universo original, o teto é **208/213 = 97,65%**; superar 98% exigiria 209 detecções. Nenhum mutante foi ignorado, nenhum operador excluído e nenhuma fonte alterada para elevar o score. A meta permanece pendente de decisão do professor sobre equivalentes ou revisão do projeto/meta.

7. Conclusão

Cobertura mede execução; mutação avalia a capacidade de detectar alterações. A melhoria demonstrou o valor de fronteiras, entradas variadas e asserções específicas. Os equivalentes também mostram que uma pontuação deve ser interpretada com o domínio e as evidências, sem transformar um alvo numérico em incentivo à manipulação.

Reprodução: npm ci; npm run validate:initial; npm run validate:final. JSONs, HTMLs, logs, capturas, hashes e análise detalhada: pasta evidence/ do fork. Commit dos testes: e32966943f9b.

Referências: enunciado do professor; documentação oficial StrykerJS; estados e métricas dos mutantes.